

# Column Masking & Query Performance

---

## A Diagnostic Guide (companion to the Column-Mask Performance Diagnostic notebook)

---

This guide explains how Unity Catalog column masks affect query performance, walks through the companion `Column_Masking_Performance_Diagnostic` notebook section by section, and gives concrete best practices with links to the Databricks documentation so your team can verify and improve on its own.

---

### TL;DR

---

- **A high mask count is not the same as high query cost.** A column mask only executes on the masked columns a query actually **selects**. Targeted queries pay little; `SELECT *` over a heavily-masked table pays the most.
  - **The language of the mask function is what matters.** In controlled testing a **SQL** mask added ~1% over an unmasked read, while an equivalent **Python** mask was ~17× slower. Databricks' own guidance is to **prefer SQL to Python UDFs** for masks.
  - **Python UDF masks don't run on serverless** (SQL warehouses / serverless compute) — so if your governed queries run on serverless, your masks are SQL, and masking is unlikely to be your bottleneck.
  - **The notebook finds what actually hurts** — it ranks tables and mask functions by real query time, flags the expensive kind, and reads a query plan to prove whether masking is in the critical path.
- 

### How column masks affect performance

---

**What a mask is.** A column mask is a UDF bound to a column; Unity Catalog rewrites every query so the column is wrapped in that function for users who aren't exempt. Masks (and row filters) are defined with SQL or Python UDFs — see [Row filters and column masks](#) and [SQL and Python UDFs in Unity Catalog](#).

1. **SQL vs Python is the big lever.** A SQL mask is applied natively and runs on **Photon**; in testing it cost ~1% over a plain read. A Python UDF mask runs row-by-row with JVM↔Python serialization, measured ~17× slower, and forces the query **off Photon**. The docs explicitly say *"Prefer SQL to Python UDFs. Python UDFs are less performant."*

2. **Determinism helps the optimizer, not per-row cost.** Marking a mask `DETERMINISTIC` (when its output depends only on its input) lets the optimizer reuse/rewrite it; it does **not** change per-row execution cost. Non-deterministic functions also can't be used inside aggregate arguments and block some plans.
  3. **Cost scales with masked columns selected.** Masks fire only on referenced columns, so cost grows ~linearly with how many masked columns a query selects — `SELECT *` is the worst case.
  4. **Materialized views:** a mask on a **base table** is applied at MV **refresh** and baked into stored data (not re-run per MV query); a mask on the **MV's own column** runs on every MV query.
  5. **Row filters are a different tool, not a mask substitute.** A row filter removes rows; a column mask transforms values. Don't swap one for the other to "save calls."
- 

## Running the diagnostic notebook

---

**How to run it.** Import `Column_Masking_Performance_Diagnostic (.ipynb or .sql)` into your workspace and run it on a **SQL warehouse** (Sections 1–2 join query history to lineage, which is heavy on a small serverless notebook). It is **100% read-only** — only `SELECT / EXPLAIN` against `system.*` views, creating nothing. It needs the `system.query`, `system.access`, and `system.information_schema` system schemas enabled. Start with the default 3-day window and widen once it's responsive.

**Section-by-section — what it shows and how to read it:**

| Notebook section                 | What it answers                                                        | How to read it                                                                                                                           | Learn more                                                 |
|----------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 1. Tables that hurt most         | Which tables consume the most query time, and which are masked         | <code>is_masked = true</code> and high <code>total_exec_s</code> is the only case where masking could be implicated — confirm in §3 & §5 | Query history<br>system table,<br>Lineage<br>system tables |
| 2. Mask functions that hurt most | Which mask functions sit on the heaviest workloads, and their language | Note the <code>language</code> column — <code>SQL</code> or <code>built-in</code> is cheap                                               | SQL & Python<br>UDFs                                       |
| 3. The expensive kind            | Python and/or non-deterministic mask functions                         | <b>No rows = you have none</b> → masking is not your bottleneck                                                                          | Row filters<br>and column<br>masks                         |
| 4. Slowest queries               | Where the time actually goes                                           | Look for <b>spill</b> , <b>shuffle</b> , low <b>cache %</b> , low <b>pruned_files</b>                                                    | Query profile                                              |

| Notebook section | What it answers                         | How to read it                                                                                                       | Learn more                   |
|------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------|
| 5. Read the plan | Whether masking is in the critical path | PhotonSecureView = cheap SQL mask; BatchEvalPython = costly Python mask; missing / partial stats = act on statistics | Photon, Cost-based optimizer |
| 6. Mask density  | Worst-case tables for SELECT *          | High masked_columns + SELECT * workloads = rewrite to explicit columns                                               | Row filters and column masks |

## Interpreting your results — a decision tree

1. **Section 3 returns no rows** (no Python / non-deterministic masks) → your masks are SQL/built-in and applied as Photon-native secure views (confirm in §5). **Column masking is not your bottleneck.** Move to the hot tables in §1 and use the §4 signals to find the real cause (below).
2. **Section 3 returns rows** → cross-check those functions against the hot tables in §1–2. Where a Python or non-deterministic mask sits on a hot table, that's a genuine cost — rewrite it (see best practices).
3. **High-density tables in SELECT \* workloads (§6)** → switch to explicit column lists.

## Best practices

### Masking

- **Prefer SQL to Python for mask functions** — see [Row filters and column masks](#) and [SQL and Python UDFs in Unity Catalog](#).
- **Mark masks DETERMINISTIC** when the output depends only on the input, to enable optimizer rewrites.
- **Select the columns you need**, not `SELECT *`, so masks fire only where required.
- **Use a row filter to hide rows and a column mask to redact values** — not one to optimize the other.

Example — a simple SQL mask:

```
CREATE OR REPLACE FUNCTION mask_string(v STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('admins') THEN v ELSE '***' END;
```

## Query & table performance (the usual real causes)

- **Keep statistics fresh.** The cost-based optimizer relies on them; run `ANALYZE TABLE ... COMPUTE STATISTICS`, or better, enable **Predictive optimization** to run `OPTIMIZE / ANALYZE / VACUUM` automatically. Background: **Cost-based optimizer**.
- **Improve data layout** with **liquid clustering** so queries prune more files (watch `pruned_files` in §4).
- **Diagnose a specific slow query** with the **query profile** — it shows per-operator time, spill, and shuffle so you can see the real bottleneck.
- **Right-size compute and reduce shuffle/spill** — see **Best practices for performance efficiency**.
- **Monitor continuously** with the **query history** and **lineage** system tables — the same sources this notebook uses.

---

*Performance figures cited here were measured in controlled Databricks benchmarking (5-million-row table, serverless, median of multiple runs) and validated against a large production-scale workspace. Reproduce them in your own environment with the companion notebook.*